```
import heapq

def heuristic(state, goal):
    h = 0

    for tile in range(1, 9):
        current = state.index(tile)
        target = goal.index(tile)

        current_row = current // 3
        current_col = current % 3

        target_row = target // 3
        target_col = target % 3

        h += abs(current_row - target_row) + abs(current_col - target_col)

    return h


def generate_successors(state):
    successors = []

    blank = state.index(0)

    row = blank // 3
    col = blank % 3

    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    for dr, dc in moves:
        new_row = row + dr
        new_col = col + dc

        if 0 <= new_row < 3 and 0 <= new_col < 3:
            new_state = state.copy()

            new_position = new_row * 3 + new_col

            new_state[blank], new_state[new_position] = \
                new_state[new_position], new_state[blank]

            successors.append(new_state)

    return successors


def print_state(state):
    for i in range(0, 9, 3):
        print(state[i:i + 3])
    print()


def print_path(state, parent, goal, g):
    path = []

    while state is not None:
        path.append(state)
        state = parent[tuple(state)]

    path.reverse()

    print("Solution:")

    for state in path:
        h = heuristic(state, goal)
        f = g[tuple(state)] + h

        print_state(state)
        print("g(n) =", g[tuple(state)])
        print("h(n) =", h)
        print("f(n) =", f)
        print()


def a_star(start, goal):
    open_list = []
    g = {}
    parent = {}
```

```
    closed = set()

    start_key = tuple(start)

    g[start_key] = 0
    parent[start_key] = None

    h = heuristic(start, goal)
    f = h

    heapq.heappush(open_list, (f, 0, start))

    while open_list:
        f, current_g, current = heapq.heappop(open_list)

        current_key = tuple(current)

        if current_key in closed:
            continue

        h = heuristic(current, goal)

        print_state(current)
        print("g(n) =", current_g)
        print("h(n) =", h)
        print("f(n) =", current_g + h)
        print()

        if current == goal:
            print_path(current, parent, goal, g)
            return

        closed.add(current_key)

        successors = generate_successors(current)

        for successor in successors:
            successor_key = tuple(successor)

            if successor_key in closed:
                continue

            new_g = current_g + 1

            if successor_key not in g or new_g < g[successor_key]:
                g[successor_key] = new_g
                parent[successor_key] = current

                h = heuristic(successor, goal)
                f = new_g + h

                heapq.heappush(
                    open_list,
                    (f, new_g, successor)
                )

    print("No solution exists")


start = list(map(int, input("Enter the initial state: ").split()))

goal = [1, 2, 3,
        8, 0, 4,
        7, 6, 5]

a_star(start, goal)
```

```
Enter the initial state: 2 8 3 1 6 4 0 7 5
[2, 8, 3]
[1, 6, 4]
[0, 7, 5]

g(n) = 0
h(n) = 6
f(n) = 6

[2, 8, 3]
[1, 6, 4]
[7, 0, 5]

g(n) = 1
h(n) = 5
```

```
f(n) = 6

[2, 8, 3]
[1, 0, 4]
[7, 6, 5]

g(n) = 2
h(n) = 4
f(n) = 6

[2, 0, 3]
[1, 8, 4]
[7, 6, 5]

g(n) = 3
h(n) = 3
f(n) = 6

[0, 2, 3]
[1, 8, 4]
[7, 6, 5]

g(n) = 4
h(n) = 2
f(n) = 6

[1, 2, 3]
[0, 8, 4]
[7, 6, 5]

g(n) = 5
h(n) = 1
f(n) = 6

[1, 2, 3]
[8, 0, 4]
[7, 6, 5]

g(n) = 6
h(n) = 0
f(n) = 6
```

Start coding or generate with AI.